Chapter 2: JavaScript and Client-Side Programming

By Er. Ukesh Thapa

Syllabus

2.1 JavaScript essentials

- 2.1.1 Data types, variables, control structures, functions
- 2.1.2 Dom manipulation, events, and form validation
- 2.1.3 Local storage and session storage
- 2.1.4 GUI interactions
- 2.1.5 JavaScript library: jQuery

2.2 Modern JavaScript (ES6+)

- 2.2.1 Arrow functions, destructuring, spread/rest operators
- 2.2.2 Callbacks, promises, async/await
- 2.2.3 Modules and imports

Syllabus

2.3 Client-side applications

2.3.1 Client-side web application development using React JS library

2.3.2 Traditional multi-page apps vs. single-page apps (SPAs)

2.3.3 Component-based UI and props

2.3.4 State management and data flow

2.3.5 Client-side routing and navigation without reloads

2.3.6 Fetch API: Fetching, displaying data, handling errors and loading states

2.3.7 Comparison of modern JavaScript libraries: React, Angular, Vue

Hours: 12

Marks Distribution: 16

Javascript (JS)

- Also Called as DHTML (Dynamic HTML)
- It was created in 10 days in 1995. It was named "JavaScript" purely as a marketing tactic because "Java" was popular then.
- The Browser Wars & The Birth of ECMAScript (1997):
  - The Solution: They went to a standards body called ECMA (European Computer Manufacturers Association) to write a rulebook.
- Major Updates: 2009 (ES5), 2015 (ES6 / ES2015) {The Big Bang}
- Now: Yearly updates.
  - Latest: ECMAScript 2024 (ES15)

"HTML is the skeleton. CSS is the skin. JavaScript is the brain."

Formal Definition JS

- "JavaScript is a high-level, interpreted, client-side scripting language used to create interactive and dynamic content on web pages."
- Breakdown of Keywords:
  - High-level: Easy for humans to read (close to English), handles memory automatically.
  - Interpreted: The browser reads the code line-by-line and executes it (no complex compiling step like C++).
  - Client-side: It runs on the user's computer (in the browser), not on the server.
  - Dynamic: It can change the webpage content after the page has loaded without refreshing.

JavaScript Variables and Constants

- A JavaScript variable is a container for storing data.
  - Two type of Variable
  - Var: Variables created with var are function-scoped
    - In the older version of JS we use var
      - example : var variableName = value
  - Let: Variables created with let are blocked-scoped
    - ES6 version of JS we use let
      - example : let variableName = value
- **Constants:**
  - A constant is a type of variable whose value cannot be changed.
    - Example: const variableName = value

Display with JS

- **Alert:**
  - A built-in function that displays a pop-up dialog box.
  - Syntax:
    - `alert("Your message goes here");`

- **console.log():**
  - Displays messages or variables in the browser's console.
  - Syntax:
    - `console.log(“Message in console”)`

JS: Data type

- There are two type of data type:
  - Primitive Data Types: They can hold a single simple value. String, Number, BigInt, Boolean, undefined, null, and Symbol are primitive data types.
  - Non-Primitive Data Types: They can hold multiple values. Objects are non-primitive data types.

Data Type	Description
String	Textual data. 'hello', "hello world!", etc.
Number	An integer or a floating-point number. 3, 3.234, 3e-2, etc.
BigInt	An integer with arbitrary precision. 900719925124740999n, 1n, etc.
Boolean	Any of two values: true or false.
undefined	A data type whose variable is not initialized. let a;
null	Denotes a null value. let a = null;
Symbol	A data type whose instances are unique and immutable. let value=Symbol('hello');
Object	Key-value pairs of collection of data. let student = {name: "John"};

JavaScript String

A string represents textual data. It contains a sequence of characters.

In JavaScript, strings are surrounded by quotes:

- Single quotes: 'Hello'
- Double quotes: "Hello"
- Backticks: `Hello`

JavaScript Number

The number type represents numeric values (both integers and floating-point numbers).

- Integers - Numeric values without any decimal parts. Example: 3, -74, etc.
- Floating-Point - Numeric values with decimal parts. Example: 3.15, -1.3, etc.

JavaScript Symbol

A Symbol is a unique and primitive value. This data type was introduced in ES6.

```
// two symbols with the same description

let value1 = Symbol("programiz");

let value2 = Symbol("programiz");


console.log(value1  === value2 );  // false
```

JavaScript Operator

- Here is a list of different JavaScript operators you will learn in this tutorial:
  - Arithmetic Operators
  - Assignment Operators
  - Comparison Operators
  - Logical Operators
  - Bitwise Operators
  - String Operators
  - Miscellaneous Operators

JavaScript Arithmetic Operators

Operator	Name	Example
`+`	Addition	`3 + 4 // 7`
`-`	Subtraction	`5 - 3 // 2`
`*`	Multiplication	`2 * 3 // 6`
`/`	Division	`4 / 2 // 2`
`%`	Remainder	`5 % 2 // 1`
`++`	Increment (increments by **1**)	`++5` or `5++ // 6`
`--`	Decrement (decrements by **1**)	`--4` or `4-- // 3`
`**`	Exponentiation (Power)	`4 ** 2 // 16`

JavaScript Assignment Operators

Operator	Name	Example
`=`	Assignment Operator	`a = 7;`
`+=`	Addition Assignment	`a += 5;  // a = a + 5`
`-=`	Subtraction Assignment	`a -= 2;  // a = a - 2`
`*=`	Multiplication Assignment	`a *= 3;  // a = a * 3`
`/=`	Division Assignment	`a /= 2;  // a = a / 2`
`%=`	Remainder Assignment	`a %= 2;  // a = a % 2`
`**=`	Exponentiation Assignment	`a **= 2;  // a = a**2`

JavaScript Comparison Operators

Operator	Meaning	Example
`==`	Equal to	`3 == 5` gives us `false`
`!=`	Not equal to	`3 != 4` gives us `true`
`>`	Greater than	`4 > 4` gives us `false`
`<`	Less than	`3 < 3` gives us `false`
`>=`	Greater than or equal to	`4 >= 4` gives us `true`
`<=`	Less than or equal to	`3 <= 3` gives us `true`
`===`	Strictly equal to	`3 === "3"` gives us `false`
`!==`	Strictly not equal to	`3 !== "3"` gives us `true`

JavaScript Logical Operators

Operator	Syntax	Description
`&&` (Logical AND)	`expression1 && expression2`	`true` only if both `expression1` and `expression2` are `true`
`||` (Logical OR)	`expression1 || expression2`	`true` if either `expression1` or `expression2` is `true`
`!` (Logical NOT)	`!expression`	`false` if `expression` is `true` and vice versa

JavaScript Comments

Types of JavaScript Comments

In JavaScript, there are two ways to add comments to code:

- // - Single-Line Comments
- /* */ - Multiline Comments

JavaScript Type Conversion

There are two types of type conversion in JavaScript:

- Implicit Conversion - Automatic type conversion.
  - Example: let result = "3" + 2; // output is string
  - Example: let result = "3" + true; // output is string
  - Example: let result = "3" + null; // output is string
- Explicit Conversion - Manual type conversion.
  - Number("5")
  - String(true)
  - Boolean(0)
- Display data type: **typeof(variable)**

document.body.style.background = "blue"

JS: Condition

- If….else condition

```
if(condition){
\\expression
}
else{
\\expression
}
```

- If….else condition

```
if(condition){
\\expression
}
else if (condition){
\\expression
}
else{
\\expression
}
```

JS: LOOP

For loop

```
for (initialExpression; condition; updateExpression) {

    // for loop body

}
```

Example:

1.
```
for (let i = 0; i < 3; i++) {
console.log("Hello,world!"); }
```

2.
```
const fruits = ["apple", "banana", "cherry"];

for (let i = 0; i < fruits.length; i++) {

    console.log(fruits[i]);

}
```

For While

```
while (condition) {

    // body of loop

}
```

For do while

```
do {

    // body of loop

} while(condition);
```

JS: Break and Continue

Break: The break statement terminates the loop immediately when it's encountered.

Keyword: **break**

Continue: The continue statement skips the current iteration of the loop and proceeds to the next iteration.

Keyword: **continue;**

switch...case Statement

```
switch (expression) {
    case value1:
        // code block to be executed
        break;
    case value2:
        // code block to be executed
        break;
    default:
        // code block to be executed
}
```

```
let trafficLight = "green";
let message = ""

switch (trafficLight) {
    case "red":
        message = "Stop immediately.";
        break;
    case "yellow":
        message = "Prepare to stop.";
        break;
    case "green":
        message = "Proceed or continue driving.";
        break;
    default:
        message = "Invalid traffic light color.";
}

console.log(message)

// Output: Proceed or continue driving.
```

JavaScript Function

A function is an independent block of code that performs a specific task, while a function expression is a way to store functions in variables.

Syntax:

```
function function_name(name) {

    // code

}
```

- Pass arguments
- Return type

JS: Function Expressions

In JavaScript, a function expression is a way to store functions in variables. For example,

```
// store a function in the square variable

let square = function(num) {

    return num * num;

};

console.log(square(5));

// Output: 25
```

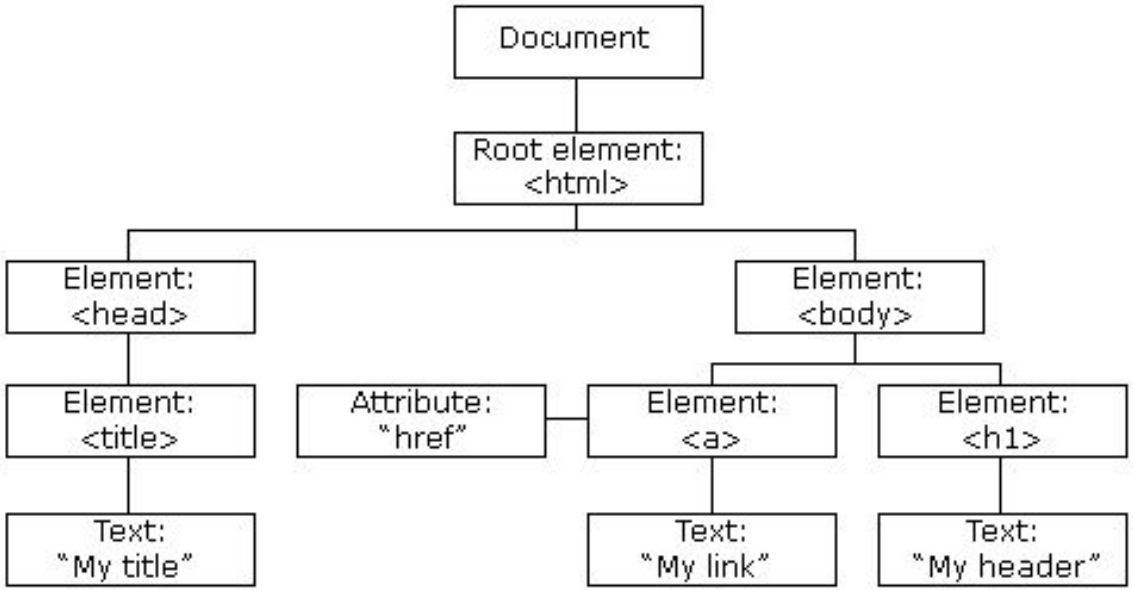
JavaScript Hoisting

In JavaScript, hoisting is a behavior in which a function or a variable can be used before declaration.

There are generally two types of hoistings in JavaScript:

- **Variable Hoisting**
  - In JavaScript, the behavior of hoisting varies depending on whether a variable is declared using var, let, or const.
- **Function Hoisting**
  - In JavaScript, function hoisting allows us to call the function before its declaration.

Document Object Model (DOM)



Document Object Model (DOM)

- JavaScript gets all the power it needs to create dynamic HTML:
  - JavaScript can change all the HTML elements in the page
  - JavaScript can change all the HTML attributes in the page
  - JavaScript can change all the CSS styles in the page
  - JavaScript can remove existing HTML elements and attributes
  - JavaScript can add new HTML elements and attributes
  - JavaScript can react to all existing HTML events in the page
  - JavaScript can create new HTML events in the page

DOM: Update UI

This is exactly what professional developers do to make a page "Dynamic."

- **Changing Content (Text & HTML)**
  - This is the most common update. You replace the words on the screen.
  - .innerText: Changes the visible text inside an element. (Safe & Fast).
  - Code: scoreDisplay.innerText = "Score: 100";
- **Changing Visuals (Styles)**
  - You can modify CSS directly from JavaScript to give immediate feedback (like turning an error message red).
  - Example: btn.style.backgroundColor = "green";
- **Changing Attributes (Properties)**
  - This changes the functionality or source of an element, not just the text.
  - Example: serImage.src = "avatar.png";
- **Visibility (Show/Hide)**
  - Making elements appear or disappear based on user logic (like closing a modal).
  - .style.display: The standard way to hide ("none") or show ("block") elements.
  - Code: modal.style.display = "none";

DOB: EventListener

- An Event Listener is a procedure in JavaScript that "**listens**" or waits for a specific event (like a click or keypress) to occur on a specific HTML element. When that event happens, it executes a defined function (callback).
- Common Types of Event Listeners
  - Mouse Events: click, mouseover, mouseout, dblclick
  - Keyboard Events: keydown, keyup
  - Form Events: submit, input
  - Window/Document Events: load, scroll
- Important Concepts
  - The Syntax Rule (No Parentheses!)
    - When adding a listener, you pass the name of the function, not the execution.
    - Correct: btn.addEventListener("click", myFunction); (Run later).
    - Wrong: btn.addEventListener("click", myFunction()); (Run now).

DOB: EventListener

- Important Concepts
  - Preventing Default Behavior (e.preventDefault)
    - Browsers have built-in behaviors (e.g., clicking a "Submit" button refreshes the page). You often need to stop this.
    - Usage: Essential for Form Handling and Single Page Applications.\
  - The Event Object (e)
    - JavaScript automatically passes a hidden argument containing details about the event.
    - **e.target:** Returns the specific HTML element that was clicked.
    - **e.key:** Returns the specific key pressed (e.g., "Enter").

Local Storage

- A storage mechanism that allows JavaScript sites and apps to save key/value pairs in a web browser with no expiration date.
- Permanent. The data stays there even if the user refreshes the page, closes the tab, or restarts the browser. It only disappears if the user manually clears their browser cache or the code specifically deletes it.
- Shared across all tabs and windows of the same browser for that specific website (domain).
- Capacity: Generally around 5MB - 10MB (huge compared to cookies).
- Why
  - User Preferences: Saving "Dark Mode" settings or font size choices.
  - Shopping Carts: Remembering items in a cart so they are still there when the user comes back 2 days later.
  - Authentication: Storing a "Keep me logged in" token (though secure cookies are often better for this).

Local Storage

```
// Saving a preference

localStorage.setItem("theme", "dark");

// Even if I restart the computer, this code works:

let currentTheme = localStorage.getItem("theme"); // Returns "dark"
```

Session Storage

- **A storage mechanism** that **saves data** only for the **duration of the session**.
- **Temporary.** The data is valid only as long as the Tab is open. The moment the user closes the tab or the browser, the data is wiped instantly.
- Isolated. **It is not shared between tabs**. If you open google.com in Tab A and google.com in Tab B, they have completely separate Session Storages.
- **Why?**
  - Sensitive Data: Banking login sessions (you want this to disappear when the user leaves).
  - If a user fills out Page 1 of a form and clicks "Next," you can save their answers in Session Storage so they don't lose them if they hit "Back."
  - Single-Session Settings: Filtering a list of products (e.g., "Sort by Price") that should reset next time they visit.

Session Storage

// Saving a temporary status

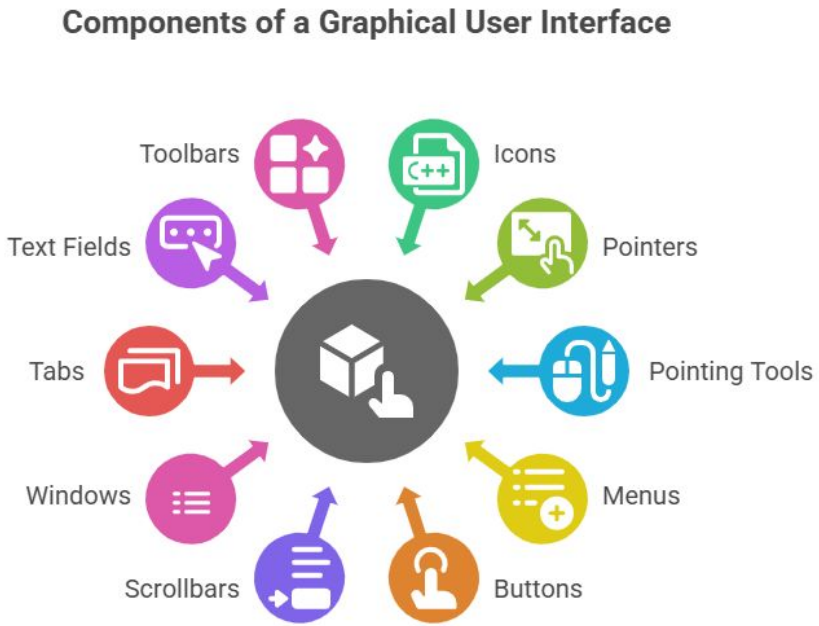
```
sessionStorage.setItem("isLoggedIn", "true");
```

// If I close the tab and reopen it:

```
let status = sessionStorage.getItem("isLoggedIn"); // Returns null (It's gone)
```

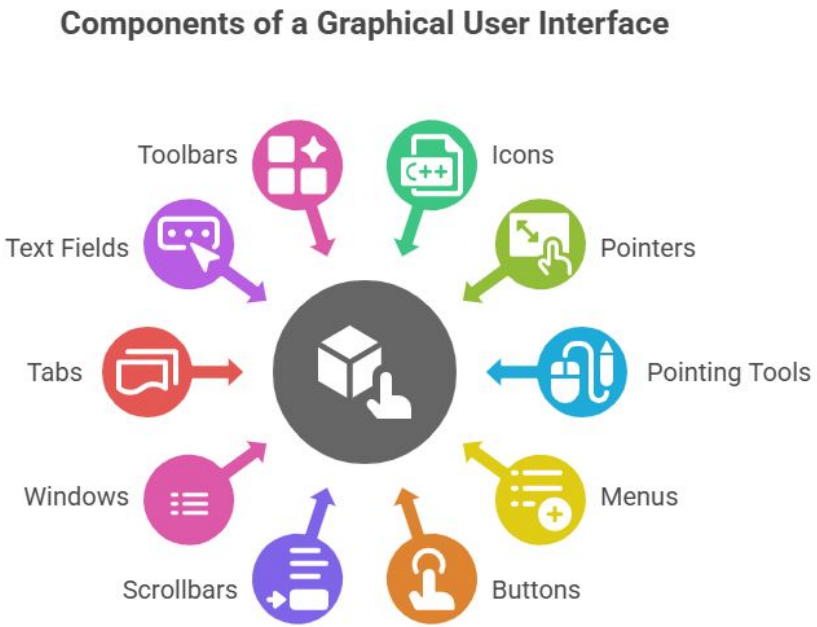
Graphical User Interface

- A Graphical User Interface (GUI) enables users to interact with digital devices through graphical elements like icons and buttons.
- **Hover Effects:**
  - Instead of just changing colors (like CSS), JavaScript allows for complex logic when the mouse moves over an element.
  - The Logic: You listen for two events: mouseover (mouse enters) and mouseout (mouse leaves).
  - Use Case: Showing a "Tooltip" (a small hint box) that follows the mouse, or triggering an animation on a completely different part of the page.



Graphical User Interface

- **Dynamic Menus (The "Hamburger" Menu):**
  - This creates navigation bars that slide in or drop down on demand, essential for mobile views.
  - The Logic: You create a button (the "trigger"). When clicked, JavaScript toggles a CSS class (like .active or .show) on the menu container.
- **Modals (Popups):**
  - A Modal is a window that forces the user to interact with it before returning to the main page.
  - The Structure: It usually has two parts:
    - Overlay: A dark, semi-transparent background that covers the whole screen.
    - Content Box: The white box floating on top (centered).

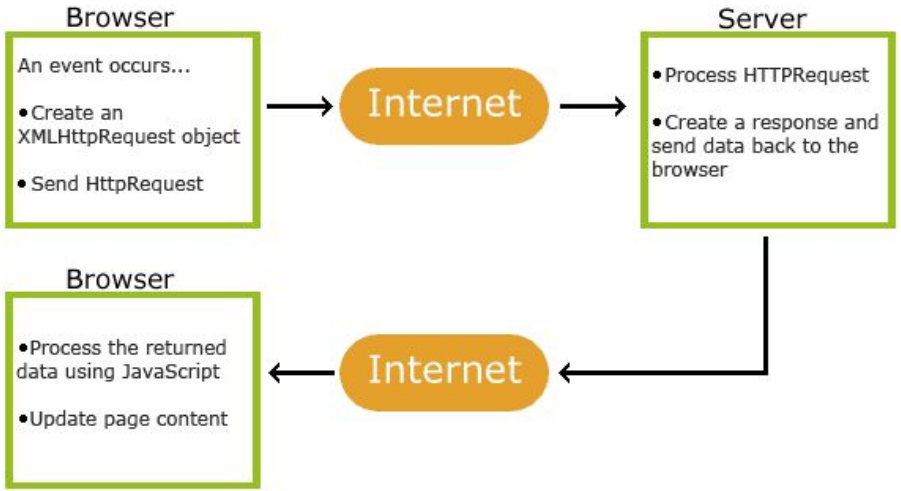


JavaScript Library: jQuery

- jQuery is a fast, small, and feature-rich JavaScript Library.
- It is designed to simplify HTML document traversal and manipulation, event handling, animation, and Ajax interactions for rapid web development.
- Its motto is "Write Less, Do More."
- **The Magic Selector ($) - DOM Shortcuts**
  - In standard JavaScript, selecting elements can be wordy. jQuery solves this with the $ (Dollar Sign) function.
    - The $ sign creates a jQuery Object that wraps the HTML element, giving it special powers.
    - Syntax: $("selector")
    - Examples:
      - Select by ID: $("#myBtn") (Instead of document.getElementById("myBtn"))
      - Select by Class: $(".error")
      - Select by Tag: $("p")

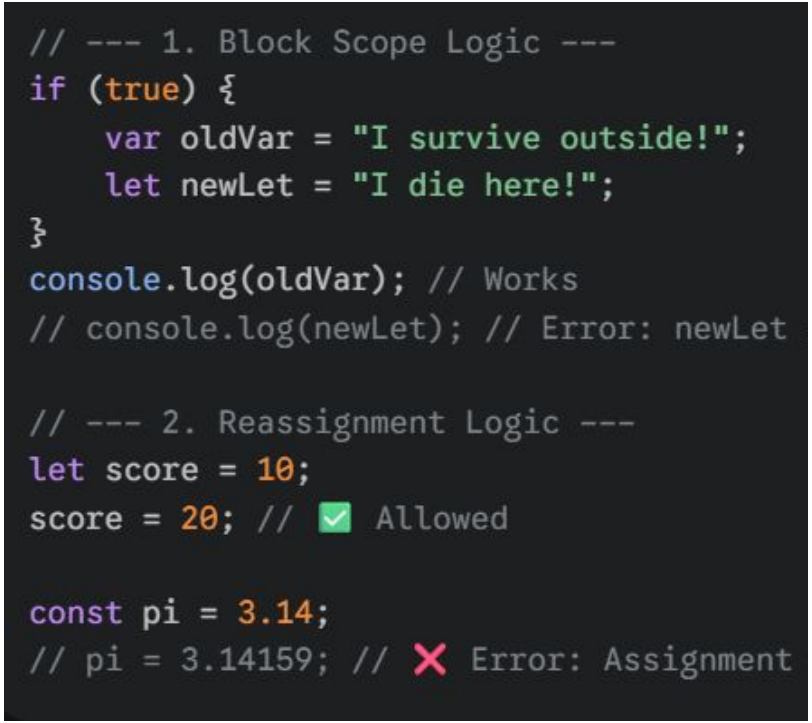
JavaScript Library: jQuery

- **AJAX Basics (No Refresh)**
  - AJAX stands for Asynchronous JavaScript And XML.
    - It allows the webpage to talk to the server (get data) without reloading the page.
    - jQuery's Role: Writing AJAX in raw JavaScript is difficult (requires 10+ lines). jQuery does it in 1 line.



Modern JavaScript (ES6+)

- **Let and Const (Variable Declaration):**
  - Concept: Replacing var with smarter, safer variables.
  - Formal Definition: let declares a block-scoped variable that can be reassigned. const declares a block-scoped variable that cannot be reassigned (it is constant).
  - They observe Block Scope.
  - This means if you define them inside an if block or loop { ... }, they do not exist outside of it. var would leak out; these do not.



Modern JavaScript (ES6+)

- **Arrow Functions (=>):**
  - An Arrow Function is a compact alternative to a traditional function expression that does not have its own bindings to this or super.
  - If the function has only one line of code, you can remove the {} and the return keyword. This is called an Implicit Return.

```
// 1. Old Way (ES5)
const add = function(a, b) {
    return a + b;
};

// 2. Modern Way (ES6 Arrow)
// Removed 'function' keyword
const addModern = (a, b) => {
    return a + b;
};

// 3. The "One-Liner" (Professional Style)
// Removed braces {} and 'return' keyword
const addPro = (a, b) => a + b;
```

Modern JavaScript (ES6+)

- **Destructuring**
  - Destructuring assignment is a syntax that allows you to "unpack" values from arrays, or properties from objects, into distinct variables.
  - It saves you from writing repetitive code

```
const colors = ["Red", "Green", "Blue"];

// Extract first two colors
const [first, second] = colors;
console.log(first); // "Red"
```

Array Destructuring

```
const user = { name: "Alex", age: 25, country: "Nepal" };

// Old Way:
// let name = user.name;
// let age = user.age;

// Modern Way:
const { name, age } = user; // Creates variables 'name' and 'age'
console.log(name); // "Alex"
```

Object Destructuring

Spread and Rest Operators (...)

- **Spread Operator (Expands)**
  - Expands an array into individual elements.
  - Combining arrays or passing array values into a function.
- **Rest Parameter (Collects)**
  - Collects multiple elements and condenses them into a single array.
  - When a function needs to accept an unlimited number of arguments.

```
const fruits = ["Apple", "Banana"];
const veggies = ["Carrot", "Potato"];

// Combine them easily
const food = [...fruits, ...veggies];
console.log(food); // ["Apple", "Banana",
```

```
// The function doesn't know how many numbers are coming
function sumAll(...numbers) {
    // 'numbers' is now an array: [10, 20, 30]
    return numbers.reduce((total, num) => total + num);
}

console.log(sumAll(10, 20, 30)); // 60
```

Why Asynchronous JavaScript?

- In standard programming (Synchronous), line 2 waits for line 1 to finish.
- But in Web Development, some tasks take time (like downloading a file or fetching data).
- We cannot freeze the whole website while waiting.
- Synchronous vs. Asynchronous
  - **Synchronous:** In synchronous code, the browser reads your code from top to bottom. If line 2 takes 5 seconds, Line 3 cannot run until Line 2 is finished.
  - **Asynchronous:** , JavaScript offloads heavy tasks (like downloading a file or a timer) to the browser's background APIs and keeps running the rest of the code.

Note: If you write a Synchronous API call (which is possible but bad practice), your website will crash/freeze for the user every time they click a button. You must use Async (Promises/Async-Await) for anything that takes time.

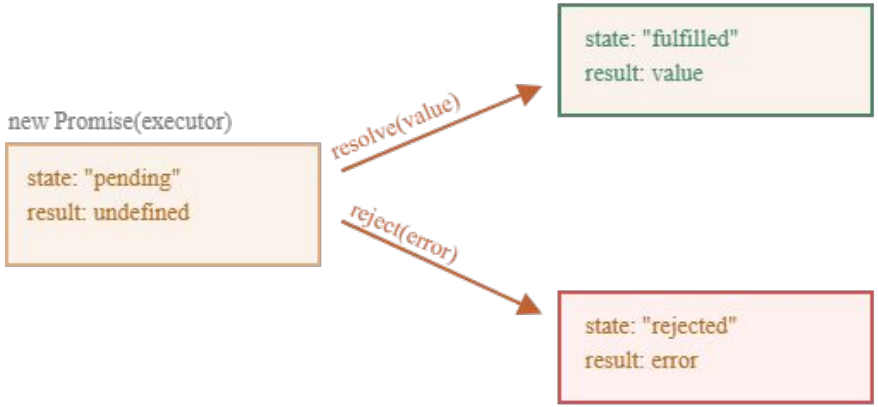
The Evolution of Async Code

- **Stage 1: Callbacks (The Old Way)**
  - Passing a function into another function to run "when you are done."
  - A function passed as an argument to another function.
  - If you have to do 3 things in a row (Login -> Get Data -> Save Data), you end up nesting callbacks inside callbacks. This is called "**Callback Hell**".

```
asyncOperation1(function(result1) {
  asyncOperation2(result1, function(result2) {
    asyncOperation3(result2, function(result3) {
      // More nested callbacks ...
    });
  });
});
```

The Evolution of Async Code

- **Stage 2: Promises (The Improvement)**
  - An object that represents a future event.
  - A Promise is an object that represents the eventual completion (or failure) of an asynchronous operation.
  - 3 States:
    - Pending: Working on it...
    - Resolved (Fulfilled): Success!
    - Rejected: Failed!



The Evolution of Async Code

- **Stage 3: Async / Await (The Modern Standard)**
  - It makes async code look like normal synchronous code.
    - **async:** Put this in front of a function to force it to return a Promise.
    - **await:** Put this in front of a promise to pause the code until the promise is resolved.

Modules and Imports

- Rather than having code written in on single js file, we have to make it modular which is called **modules.**
  - **Export:** You explicitly choose what parts of a file are shared with others.
  - **Import:** You explicitly choose what you want to use in your main file.
- It is also worth noting that:
  - Some tools may never support .mjs.
  - The **<script type="module">** attribute is used to denote when a module is being pointed to, as you'll see below.
- **Two Types of Exports:**
  - Named Export (Specific)
    - Use when: You want to export multiple things (functions, variables) from one file.
    - Rule: You must use the exact same name when importing using curly braces { }.
  - Default Export (Main)
    - Use when: The file does only one main thing (like a single class or a main function).
    - Rule: You can name it whatever you want when importing (no curly braces).

Modules and Imports: Sample

Syntax:

Type	Exporting	Importing
Named	export const tax = 0.13;	import { tax } from './file.js';
Default	export default User;	import User from './file.js';
Both	(You can have both in one file)	import User, { tax } from './file.js';

JavaScript XML (JSX)

- JSX is a syntax extension for JavaScript that looks like HTML. It allows you to write HTML structures directly inside JavaScript code.
- Important Points:
  - It is not HTML: Browsers cannot understand JSX.
  - Tools like **Babel or SWC** must convert it into regular JavaScript (React.createElement) before the browser runs it.
  - **CamelCase Attributes:** Since it is JavaScript, you cannot use standard HTML attribute names if they conflict with JS keywords. **class → className**
  - You can insert dynamic variables inside curly braces {}.
  - A component must return one wrapping element (**or a Fragment <>...</>**). You cannot return two sibling s without a wrapper.

Components

- Components are independent, reusable pieces of code.
- They serve as the "**building blocks**" of a React application.
- Think of them like Lego bricks; you combine small blocks (Button, Navbar) to build a castle (App).
- **Important Points:**
  - **Capitalization is Mandatory:** React distinguishes components from HTML tags by the first letter.
  - You write the code once (e.g., a Button component) and use it in 50 different places.
  - Components receive data via "props" (properties), similar to how functions receive arguments.
  - A component should not change its inputs (props). It must always return the same JSX for the same props.

Functional Components vs Class Components

Feature	Functional Components	Class Components
Syntax	Simple function with Hooks	ES6 classes extending React.Component.
State Management	Requires Hooks like useState	Managed via this.state and this.setState.
Lifecycle Methods	Managed via useEffect	Dedicated lifecycle methods like componentDidMount.
Performance	Slightly faster due to fewer abstractions	Slightly slower due to more internal logic.

Rendering

- Rendering is the process where React turns your Components (code) into actual DOM nodes (visible UI) on the webpage.
- The Virtual DOM: React does not update the real browser directly (which is slow). It updates a "Virtual" copy first.
- React creates a new Virtual DOM when data changes.
- It compares (diffs) it with the old Virtual DOM.
- It updates only the parts of the Real DOM that actually changed.
- **Triggering a Re-render:** A component will only re-render (update) if:
  - Its State changes.
  - Its Props change.
  - Its Parent re-renders.

Multi-Page App (MPA) vs Single Page App (SPA)

Feature	Multi-Page App (MPA)	Single Page App (SPA)
Page Load	Reloads entirely on every click.	Loads once, then updates dynamically.
Speed	Slower (Server sends full HTML).	Faster (Server sends tiny JSON). reload-free navigation
User Experience	"Flashes" between pages.	Smooth, app-like feel.
Tech Stack	PHP, Django Templates, HTML/CSS.	React, Vue, Angular.
Feature	**Multi-Page App (MPA)**	**Single Page App (SPA)**

Props (Properties)

- Props are read-only inputs that a parent component passes down to its children to customize them.
- Think of a Component like a Function and Props like Arguments.
- The "Golden Rules" of Props
  - Read-Only (Immutable): A child component cannot change its own props.
  - One-Way Flow: Props always travel Down (Parent → Child). They never travel Up.
  - Any Data Type: You can pass Strings, Numbers, Arrays, Objects, and even Functions (like onClick) as props.
- prop drilling:

Props: Data Handling

- Pass Multiple Properties
- Different Data Types
- Object Props
- Array Props
- Pass Props from Component to Component
- Destructuring
- Props Children

Note: Example for each methods

React Hooks

- Hooks are functions that let you "hook into" React state and lifecycle features from functional components.
- Hook Rules
  - Hooks can only be called inside React function components.
  - Hooks can only be called at the top level of a component.
  - Hooks cannot be conditional
- Custom Hooks
  - If you have stateful logic that needs to be reused in several components, you can build your own custom Hooks.

React Hook: useState

- useState is a React Hook that allows a Functional Component to have "Memory."
- It preserves data between renders and triggers a UI update (re-render) whenever that data changes.
- Return Value: It always returns an array with exactly two elements: The Current Value, The Setter Function
- Syntax
  - **const [count, setCount] = useState(0);**
  - count (The Variable): This holds the current data. You read from here.
  - setCount (The Setter): This is a function. You use this to update the data.
  - useState(0) (The Initializer): This is the starting value (e.g., 0, "Hello", [], or false).

React Hook: Custom Hooks

- Custom Hooks are the most powerful feature in React for writing clean, professional code.
- If you find yourself writing the same useEffect or useState logic in two different components, you should extract that logic into a Custom Hook.
- A Custom Hook is just a JavaScript function, but its name must start with use (e.g., useFetch, useForm, useLocalStorage).
- Applications:
  - Start a loading spinner.
  - Fetch data from an API/Database.
  - Stop the loading spinner.
  - Handle errors if the API fails.

useEffect

- useEffect is a React Hook used to manage Side Effects in functional components.

  useEffect(<function>, <dependency>)

- What is a Side Effect? Any operation that affects something outside the scope of the function returning the JSX.
- It runs after the render is committed to the screen
- Ways:
  - No dependency passed
  - An empty array
  - Props or state values
  - Cleanup Function

React Hook: Custom Hooks

- Importance:
  - Reusability
  - Clean Code
  - Isolation
- Common Custom Hooks Professionals Use:
  - useFetch: API calls.
  - useForm: Handling complex form inputs and validation.
  - useLocalStorage: Saving data to the browser so it stays after refresh.
  - useOnClickOutside: Detecting clicks outside a modal (to close it).
  - useAuth: Checking if a user is logged in.

ReactJS Unidirectional Data Flow

Aspect	Unidirectional Data Flow	Bidirectional Data Flow
Direction of Data	Parent-to-Child	Both Parent-to-Child and Child-to-Parent
Control	Centralized in parent components	Shared between parent and child components
Complexity	Easier to debug and manage	Requires careful synchronization of data
Example Frameworks	ReactJS, Redux, MobX	Angular (via Two-Way Data Binding)

React Router Basics

- The standard library for implementing navigation in React single-page applications (SPAs) without requiring full page reloads.
- Routing means handling navigation between different views.
- The primary components for basic routing with react-router-dom
  - BrowserRouter
  - Routes
  - Route
  - Link and NavLink
  - Outlet
- Install : npm install react-router-dom
- There is a special version of the Link component called NavLink that knows whether the link's URL is "active" or not.
  - Navigation menus
  - Breadcrumbs
  - Tabs

Fetch API

- Fetching (fetch)
  - React doesn't know about your database. You must use useEffect to trigger a "side effect" that goes out to the internet to grab data.
  - Real API: Connects to your actual backend (e.g., localhost:5000/api/students or Firebase).
  - Placeholder API: Services like JSONPlaceholder allow you to test your frontend code (tables, cards, loaders) before your backend is even built. It mimics a real server perfectly.
- Loading States
  - The internet has "latency" (delay). If you don't handle this, the user sees a blank white screen for 1-2 seconds.
- Handling Errors
  - fetch() only throws an error if the user has No Internet. If the server replies "404 Not Found", fetch thinks that is a "Success"
- Displaying Data
  - Transforming Raw Data (JSON) into UI (JSX).
  - .map() loops through the array.

Comparative analysis on React vs Vue vs Angular

Feature	React	Vue	Angular
Owner	Meta (Facebook)	Open Community	Google
Philosophy	It gives you the UI blocks. You choose the router, state management, etc.	Can be used simply (like jQuery) or fully (like Angular).	Everything is included (Router, HTTP, Validation) and configured for you.
Syntax	**JSX (JavaScript XML):** Mixes HTML & JS in one file. Requires strong JS knowledge.	**HTML Templates:** Separates HTML, JS, and CSS. Very readable.	**TypeScript (Required):** Strict typing, Decorators, OOP style.
Learning Curve	Easy to start, but hard to master the ecosystem (Redux, Router, etc.).	The most beginner-friendly. Reads like standard web code.	Requires learning TypeScript, RxJS, Modules, and Dependency Injection.
Data Binding	**One-Way (Down):** Parent → Child. Predictable and easy to debug.	**Two-Way (Optional):** Mostly one-way, but allows two-way for forms.	**Two-Way (Bi-directional):** UI updates Logic, Logic updates UI automatically.
Performance	Fast updates via diffing algorithms.	Fast and lightweight (often slightly lighter than React).	Historically slower, but heavily optimized in recent versions (Ivy Compiler).
Scalability	High	Medium/High	Very High